

SymPy Bug Report

1 Bug 59: Right Limit of Digamma at Zero Returns Zoo Instead of Negative Infinity

1.1 Minimal Reproducer

```
from sympy import limit, polygamma, symbols

x = symbols("x")
expr = polygamma(0, x)

print(limit(expr, x, 0, dir="+"))
```

1.2 Observed Behavior

[illegible]

The symbolic one-sided limit returns `zoo`, SymPy’s complex-infinity marker, even though nearby positive real samples are large negative real numbers and agree with the independent `mpmath.digamma` value. The result therefore loses the sign of a real directional pole.

1.3 Expected Behavior

The correct value is

$$\lim_{x \rightarrow 0^+} \text{polygamma}(0, x) = \lim_{x \rightarrow 0^+} \psi(x) = -\infty.$$

Equivalently, the SymPy call should return `S.NegativeInfinity`, displayed as $-\infty$, not `zoo`.

1.4 Mathematical Explanation

The order-zero polygamma function is the digamma function,

$$\text{polygamma}(0, x) = \psi(x) = \frac{\Gamma'(x)}{\Gamma(x)}.$$

At the origin it has the Laurent expansion

$$\psi(x) = -\frac{1}{x} - \gamma + O(x), \quad x \rightarrow 0,$$

where γ is Euler’s constant. Along the right-hand real approach $x \rightarrow 0^+$, the principal part $-1/x$ tends to $-\infty$. Thus the one-sided limit is a signed real infinity. Returning `zoo` is not an equivalent representation: `zoo` denotes an undirected complex infinity and discards the directional sign that the real limit determines.

1.5 Source-Level Diagnosis

The diagnosis identifies the missing local pole expansion in `sympy/functions/special/gamma_functions.py`, specifically in the leading-term method for `polygamma` around line 790. The traced call path is that `limit()` invokes `Limit.doit()` in `sympy/series/limits.py`; the meromorphic/leading-term path cannot obtain a usable leading term for `polygamma(0, x)`, and `gruntz()` also fails while trying to expand `polygamma(0, 1/x)`. `Limit.doit()` then falls back to the heuristic path in `sympy/series/limits.py`, which evaluates the function at the argument limit and constructs `polygamma(0, 0)`.

The key source-level behavior is that the `polygamma` leading-term method returns the original expression unchanged for this finite pole, instead of returning the principal part $-1/x$. Since the returned expression still contains x , `Expr.leadterm()` rejects it as a leading coefficient. After the leading-term and `gruntz()` routes fail, `polygamma.eval()` in `sympy/functions/special/gamma_functions.py` represents `polygamma(0, 0)` as `S.ComplexInfinity`. That fallback endpoint evaluation explains why the final limit is `zoo` rather than the signed value $-\infty$.

```
# Diagnosed behavior, paraphrased from the source trace:
# polygamma._eval_as_leading_term() leaves polygamma(0, x)
# unchanged at the finite pole instead of exposing -1/x.
# The limit fallback then evaluates polygamma(0, 0), which is zoo.
```

A plausible fix direction identified by the diagnosis is to teach the `polygamma` leading-term method, or the finite-point series machinery, about poles at nonpositive integers. For order 0 at $z = 0$, the leading term should be $-1/z$; more generally, the implementation should expose the signed principal part at $z = -k$ so directional limits can preserve the sign.

1.6 Additional Failing Instances

The independent verification checks the positive rescaling family

$$\lim_{x \rightarrow 0^+} \text{polygamma}(0, ax), \quad a \in \{1, 2, 3, 4, 5, 6\}.$$

For every positive scale a , the same Laurent expansion gives

$$\psi(ax) = -\frac{1}{ax} - \gamma + O(x), \quad x \rightarrow 0^+,$$

so each limit is $-\infty$. The verification records that SymPy instead returns `zoo` throughout this family.

Input / Expression	SymPy behavior	Expected behavior
$\lim_{x \rightarrow 0^+} \text{polygamma}(0, x)$	zoo	$-\infty$
$\lim_{x \rightarrow 0^+} \text{polygamma}(0, 2x)$	zoo	$-\infty$
$\lim_{x \rightarrow 0^+} \text{polygamma}(0, 3x)$	zoo	$-\infty$
$\lim_{x \rightarrow 0^+} \text{polygamma}(0, 4x)$	zoo	$-\infty$
$\lim_{x \rightarrow 0^+} \text{polygamma}(0, 5x)$	zoo	$-\infty$
$\lim_{x \rightarrow 0^+} \text{polygamma}(0, 6x)$	zoo	$-\infty$

1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check. The closest related issues were SymPy issue 5415, an older `limit` failure involving `polygamma` for a different expression and a limit at infinity, and a comment on issue 5229 discussing `polygamma(0, x)` in harmonic-number asymptotics. These indicate that `polygamma` limits are a known problem area. Critically, this specific right-hand zero-pole sign error `limit(polygamma(0, x), x, 0, dir='`+') = zoo` instead of  $-\infty$  IS publicly reported and fixed: open (unmerged) SymPy PR #29424 (<https://github.com/sympy/sympy/pull/29424>, created 2026-03-16) targets exactly this symptom and root cause (`polygamma._eval_as_leading_term` in `gamma_functions.py`). The candidate therefore duplicates an existing unmerged fix rather than being a novel specific bug, though it remains actionable as confirmation of a still-open problem.

1.8 Proposed SymPy Regression Test

```
from sympy import S, limit, polygamma, symbols
import pytest

@pytest.mark.parametrize("scale", [1, 2, 3, 4, 5, 6])
def test_digamma_right_limit_at_zero_preserves_sign(scale):
    x = symbols("x")
    assert limit(polygamma(0, scale*x), x, 0, dir="+") == S.NegativeInfinity
```